

BlockSupply — Decentralized Supply Cloud

Project Overview

BlockSupply is a Supply Chain Management Dashboard built with Flask, deployed on AWS to demonstrate core cloud computing concepts. The application manages inventory, shipments, and workflows with role-based access control, backed by six AWS services.

Team: Rehman Khan (150096724114)

Tech Stack

Layer	Technology	Purpose
Backend	Flask 3.0, Python 3.11	Web framework & API
ORM	Flask-SQLAlchemy 3.1	Database abstraction
Auth	Flask-Login 0.6	Session management & RBAC
Database	MariaDB (Amazon RDS)	Relational data storage
Storage	Amazon S3 (boto3)	Report file storage
Monitoring	Amazon CloudWatch	Metrics & logging
Compute	Amazon EC2 (Amazon Linux 2023)	Application hosting
Networking	Amazon VPC	Network isolation
Security	AWS IAM	Role-based AWS access
WSGI Server	Gunicorn 22.0	Production server

Features

Role-Based Access Control

Three user roles with different permissions:

- **Admin** — Full access, approves workflows, manages all resources

- **Manager** — Can create products, shipments, and tasks (tasks require admin approval)
- **Staff** — Can view data and update quantities

Demo accounts: `admin/admin123` · `manager/manager123` · `staff/staff123`

Inventory Management

- Add products with SKU, quantity, warehouse assignment
- Update stock quantities
- Low-stock alerts (when quantity $\leq$ reorder level)
- Multi-warehouse tracking

Shipment Tracking

- Create shipments with source/destination
- Status tracking: Pending → In Transit → Delivered / Cancelled
- Linked to products for traceability

Workflow Management

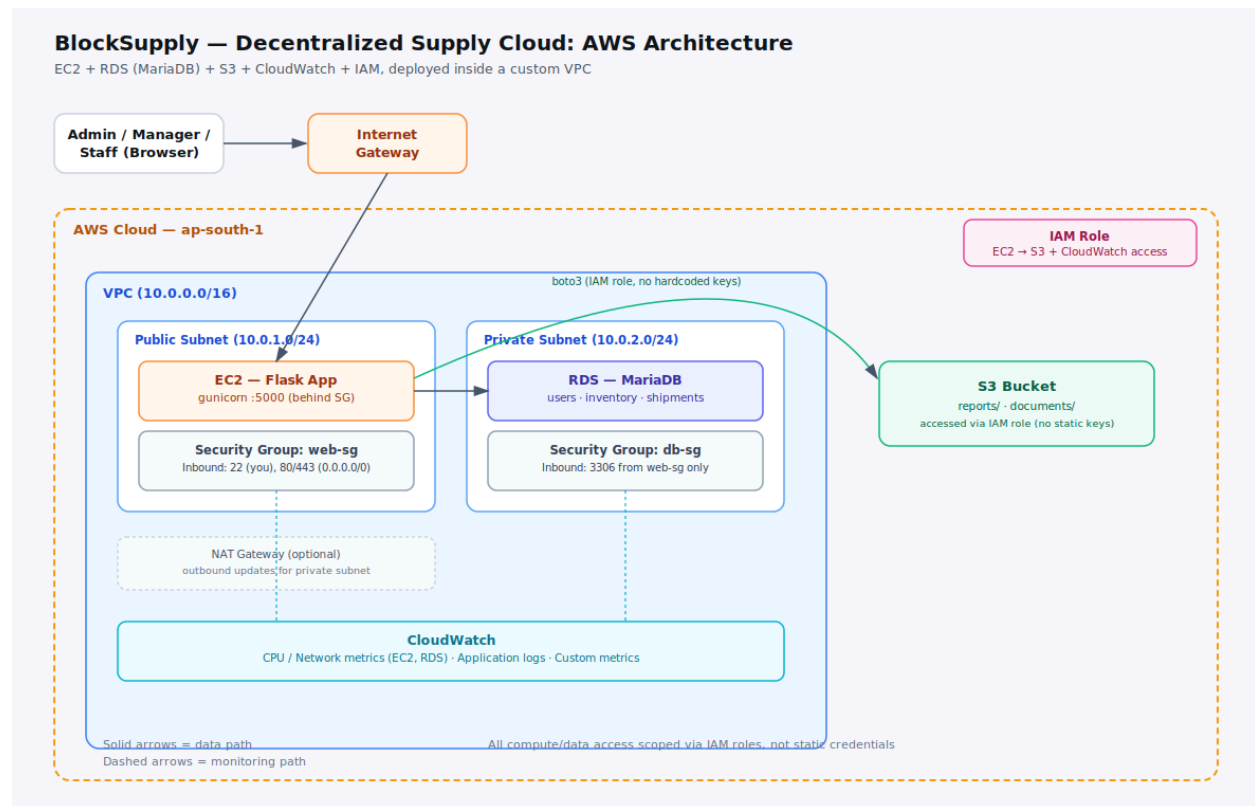
- Task creation with assignment and due dates
- Manager → Admin approval chain
- Status lifecycle: Open → In Progress → Pending Approval → Approved/Rejected → Done

Reporting Dashboard

- Inventory metrics by warehouse
- Shipment status analytics
- CSV export to S3
- Low-stock product alerts

AWS Architecture

Architecture Diagram



AWS Services Used

1. Amazon VPC

- **Purpose:** Network isolation and security
- **Config:** CIDR `10.0.0.0/16`, 1 public subnet, 1 private subnet
- **Key Detail:** No NAT Gateway used (saves ~\$32/month)

2. Amazon EC2

- **Purpose:** Hosts the Flask application
- **Instance:** `t3.micro` (free tier eligible)
- **AMI:** Amazon Linux 2023
- **Subnet:** Public (with auto-assigned public IP)
- **Security Group (web-sg):** Inbound SSH (22), HTTP (80), TCP (5000)

3. Amazon RDS (MariaDB)

- **Purpose:** Relational database for all application data
- **Instance:** `db.t3.micro`, Single-AZ (free tier eligible)
- **Storage:** 20 GB gp2
- **Subnet:** Private (not publicly accessible)
- **Security Group (`db-sg`):** Inbound MySQL (3306) from `web-sg` only

4. Amazon S3

- **Purpose:** Stores exported CSV inventory reports
- **Bucket:** `blocksupply-reports-rehman0601`
- **Access:** Private (no public access), accessed via IAM role
- **Key Detail:** No hardcoded AWS credentials — uses instance role

5. AWS IAM

- **Purpose:** Grants EC2 access to S3 and CloudWatch without static credentials
- **Role:** `blocksupply-ec2-role`
- **Policies:** `AmazonS3FullAccess`, `CloudWatchAgentServerPolicy`
- **Key Detail:** Demonstrates least-privilege principle — EC2 assumes the role automatically

6. Amazon CloudWatch

- **Purpose:** Monitoring and observability
 - **Infrastructure Metrics:** CPU utilization, network I/O (automatic, no setup)
 - **Custom Metrics (via boto3):**
 - `UserLogin` — pushed on each successful login
 - `ReportExported` — pushed on each CSV export
 - **Namespace:** `BlockSupply/App`
-

Database Design

Entity-Relationship Overview

Six tables with foreign key relationships:

```
users ─┬─> tasks (assigned_to, created_by)
      └─> approval_chains (approver_id)

warehouses ─> products (warehouse_id)

products ─> shipments (product_id)

tasks ─> approval_chains (task_id)
```

Tables

users

Column	Type	Constraints
id	INT	PRIMARY KEY, AUTO_INCREMENT
username	VARCHAR(80)	UNIQUE, NOT NULL
password_hash	VARCHAR(255)	NOT NULL
role	VARCHAR(20)	NOT NULL, DEFAULT 'staff'
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP

warehouses

Column	Type	Constraints
id	INT	PRIMARY KEY, AUTO_INCREMENT
name	VARCHAR(120)	NOT NULL
location	VARCHAR(200)	—

products

Column	Type	Constraints
id	INT	PRIMARY KEY, AUTO_INCREMENT

Column	Type	Constraints
name	VARCHAR(150)	NOT NULL
sku	VARCHAR(60)	UNIQUE, NOT NULL
quantity	INT	DEFAULT 0
reorder_level	INT	DEFAULT 10
warehouse_id	INT	FK → warehouses(id)
updated_at	DATETIME	AUTO-UPDATE

shipments

Column	Type	Constraints
id	INT	PRIMARY KEY, AUTO_INCREMENT
product_id	INT	FK → products(id)
quantity	INT	NOT NULL
source	VARCHAR(150)	NOT NULL
destination	VARCHAR(150)	NOT NULL
status	VARCHAR(30)	DEFAULT 'Pending'
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP
updated_at	DATETIME	AUTO-UPDATE

tasks

Column	Type	Constraints
id	INT	PRIMARY KEY, AUTO_INCREMENT
title	VARCHAR(200)	NOT NULL
description	TEXT	—
assigned_to	INT	FK → users(id)
created_by	INT	FK → users(id)
status	VARCHAR(30)	DEFAULT 'Open'
due_date	DATE	—
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP

approval_chains

Column	Type	Constraints
id	INT	PRIMARY KEY, AUTO_INCREMENT
task_id	INT	FK → tasks(id)
approver_id	INT	FK → users(id)
status	VARCHAR(20)	DEFAULT 'Pending'
comment	VARCHAR(300)	—
decided_at	DATETIME	—

Indexes

- `idx_products_warehouse` — products by warehouse
- `idx_shipments_status` — shipments by status
- `idx_shipments_product` — shipments by product
- `idx_tasks_status` — tasks by status
- `idx_tasks_assignee` — tasks by assigned user

Project Structure

```
BlockchainSupply/
├─ app.py                # Flask application & routes
├─ models.py             # SQLAlchemy ORM models
├─ config.py             # Environment-based configuration
├─ schema.sql            # MariaDB DDL
├─ requirements.txt       # Python dependencies
├─ .env.example          # Template for environment variables
├─ templates/            # Jinja2 HTML templates
│   ├─ base.html         # Layout with sidebar navigation
│   ├─ login.html        # Authentication page
│   ├─ dashboard.html    # Overview with metrics
│   ├─ inventory.html    # Product & warehouse management
│   ├─ shipments.html    # Shipment tracking
│   ├─ workflow.html     # Task & approval management
│   └─ reports.html      # Analytics & CSV export
├─ static/
│   └─ style.css         # Application stylesheet
└─ deploy/
```

```
|— DEPLOYMENT_GUIDE.md      # Step-by-step AWS setup
|— PRICING_ANALYSIS.md      # Cost breakdown
|— architecture-diagram.svg / .png
|— blocksupply.service      # systemd unit for gunicorn
```

Deployment Details

EC2 Instance

- **IP Address:** 52.54.56.120
- **OS:** Amazon Linux 2023
- **Python:** 3.11
- **App Server:** Gunicorn (3 workers, port 5000)
- **Process Manager:** systemd (`blocksupply.service`)
- **Auto-restart:** Enabled (restarts on crash after 5 seconds)

RDS Instance

- **Endpoint:** `database-1.c0pu4qi42w28.us-east-1.rds.amazonaws.com`
- **Engine:** MariaDB
- **Database Name:** `blocksupply`
- **Port:** 3306
- **Access:** Private subnet, only accessible from EC2 via `db-sg`

S3 Bucket

- **Name:** `blocksupply-reports-rehman0601`
- **Region:** us-east-1
- **Structure:** `reports/` — contains exported CSV files
- **Access:** IAM role only (no public access)

Security Groups

web-sg (EC2):

Rule	Port	Source
SSH	22	My IP
HTTP	80	0.0.0.0/0
Custom TCP	5000	0.0.0.0/0

db-sg (RDS):

Rule	Port	Source
MySQL/Aurora	3306	web-sg

Environment Variables

Variable	Description	Example
SECRET_KEY	Flask session secret	secret_secret
DB_USER	RDS master username	admin
DB_PASSWORD	RDS master password	
DB_HOST	RDS endpoint	database-1.xxx.us-east-1.rds.amazonaws.com
DB_PORT	MariaDB port	3306
DB_NAME	Database name	blocksupply
S3_BUCKET	S3 bucket for reports	blocksupply-reports-rehman0601
AWS_REGION	AWS region	us-east-1
CLOUDWATCH_NAMESPACE	CloudWatch namespace	BlockSupply/App

Pricing Analysis

Cost Breakdown

Component	Free Tier (first 12 months)	After Free Tier
EC2 (t2.micro)	\$0	~\$9–10/mo
EBS Storage (20 GB)	\$0	~\$2/mo
RDS (db.t3.micro)	\$0	~\$24/mo
S3 (< 1 GB)	\$0	~\$0.03/mo
CloudWatch	\$0	~\$0.30/mo
IAM	\$0	\$0 (always free)
VPC (no NAT GW)	\$0	\$0
Total	\$0/month	~\$33–36/month

Cost Optimization Decisions

1. **No NAT Gateway** — saves ~\$32/month; RDS doesn't need outbound internet
2. **Single-AZ RDS** — Multi-AZ not needed for a demo workload
3. **t2.micro / db.t3.micro** — smallest burstable instances, sufficient for low traffic
4. **IAM roles over static keys** — no credential rotation cost, better security

CloudWatch Metrics

Infrastructure Metrics (Automatic)

- **EC2:** CPUUtilization, NetworkIn, NetworkOut, DiskReadOps, DiskWriteOps
- **RDS:** CPUUtilization, DatabaseConnections, FreeStorageSpace, ReadIOPS, WriteIOPS

Custom Application Metrics

Metric Name	Trigger	Namespace
UserLogin	Each successful login	BlockSupply/App
ReportExported	Each CSV report export	BlockSupply/App

Custom metrics are pushed via `boto3` CloudWatch API using the EC2 instance's IAM role — no hardcoded credentials.

How to Run Locally

```
# Clone the repo
git clone https://github.com/rehman0601/BlockChainSupply
cd BlockChainSupply/blocksupply

# Set up Python environment
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt

# Configure environment
cp .env.example .env
# Edit .env with your database credentials

# Initialize database
export FLASK_APP=app.py
flask seed-db

# Run
python app.py
# Visit http://localhost:5000
```

Key Implementation Highlights

Security (IAM Best Practice)

The application accesses S3 and CloudWatch through an **IAM instance role** attached to EC2 — never hardcoded AWS access keys. This follows AWS security best practices:

```
# No credentials needed – boto3 auto-discovers the IAM role
s3 = boto3.client("s3", region_name="us-east-1")
s3.put_object(Bucket="...", Key="...", Body="...")
```

Network Security (VPC Design)

- EC2 in **public subnet** — accessible from internet
- RDS in **private subnet** — only accessible from EC2's security group
- Security group references (not IP-based) for inter-service access

Application Monitoring

Custom CloudWatch metrics are pushed silently — if the IAM role is missing or the app runs locally, the error is caught and logged without crashing:

```
def put_metric(metric_name, value=1, unit="Count"):
    try:
        cw = boto3.client("cloudwatch", region_name="us-east-1")
        cw.put_metric_data(Namespace="BlockSupply/App", ...)
    except Exception as e:
        logger.warning(f"CloudWatch metric push skipped: {e}")
```

Screenshots Checklist

- ☐ EC2 instance running (AWS Console)

EC2 > Instances

Instances (1/3) Info

Saved filter sets: Choose filter set, Find Instance by attribute or tag (case-sensitive)

Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DNS	Public IPv4 ...
vpc-training	i-0f91e35a7e1a1a366	Stopped	t3.micro	—	us-east-1d	—	—
frontend	i-012df606f8723098e	Stopped	t3.micro	—	us-east-1f	—	—
ply-app	i-02e8f908d5588d88e	Running	t3.micro	3/3 checks passed	us-east-1a	ec2-52-54-56-120.com...	52.54.56.120

i-02e8f908d5588d88e (blocksupply-app)

Details | Status and alarms | Monitoring | Security | Networking | Storage | Tags

Instance summary Info

Instance ID: i-02e8f908d5588d88e

Public IPv4 address: 52.54.56.120 | open address

Private IPv4 addresses: 10.0.0.120

Public DNS: ec2-52-54-56-120.compute-1.amazonaws.com | open address

Instance state: Running

Private IP DNS name (IPv4 only): ip-10-0-0-120.ec2.internal

Instance type: t3.micro

IPV6 address: —

Hostname type: IP name: ip-10-0-0-120.ec2.internal

Answer private resource DNS name: —

CloudShell | Feedback | Console Mobile App

© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

❑ RDS instance available, showing endpoint

us-east-1.console.aws.amazon.com/rds/home?region=us-east-1#database:id=database-1;is-cluster=false

AWS Aurora and RDS > Databases > database-1

database-1

Summary

DB identifier: database-1

Status: Available

Class: db.t4g.micro

Role: Instance

Engine: MariaDB

Region & AZ: us-east-1a

CPU: 2.84%

Current activity: 1 Connections

Recommendations

Connectivity & security | Monitoring | Logs & events | Configuration | Maintenance & backups | Data migrations | Tags | Recommendations

Connect using Info

Code snippets: Use when connecting through SDK, APIs, or third-party tools including agents.

CloudShell: Use for a quick access to AWS CLI that launches directly from the AWS Management Console.

Endpoints: Use when connecting through any IDE interface.

Master username: admin

Internet access gateway: Disabled

Port: 3306

Endpoint type: Instance endpoint

Additional configurations

Connectivity & security

Endpoint & port

Endpoint: database-1.c0pu4q42w28.us-east-1.rds.amazonaws.com

Port: 3306

Networking

Availability Zone: us-east-1a

VPC: blocksupply-vpc (vpc-098cca9190def8507)

Subnet group: rds-ec2-db-subnet-group-1

Security

VPC security groups: rds-ec2-1 (sg-03106826db08f9e3c)

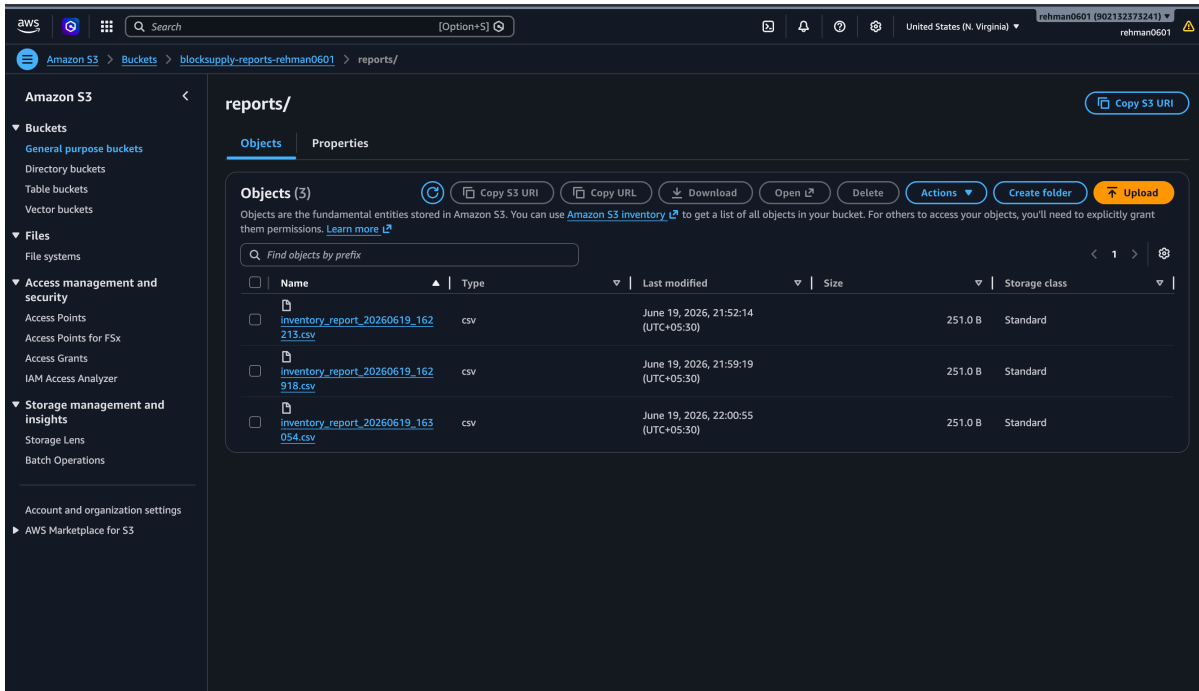
Publicly accessible: No

Certificate authority: Info

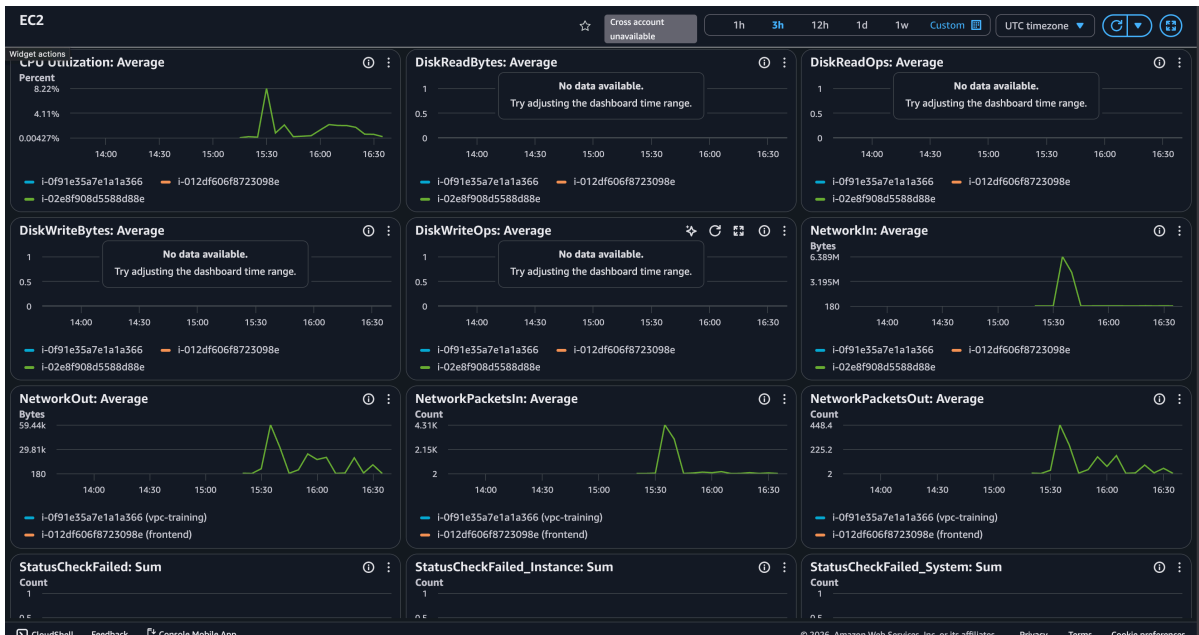
CloudShell | Feedback | Console Mobile App

© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

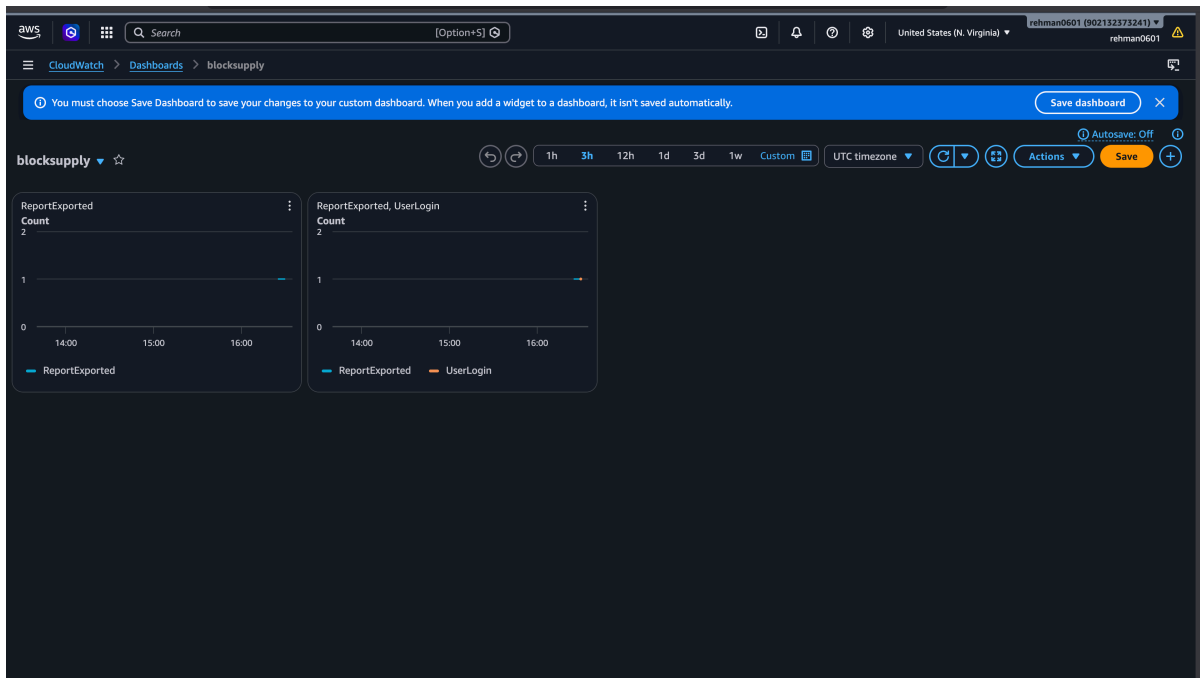
❑ S3 bucket with uploaded report CSV



☐ CloudWatch EC2 CPU/Network graph



☐ CloudWatch custom metric (UserLogin / ReportExported)



☐ Security group rules (web-sg and db-sg)

EC2

Dashboard

AWS Global View

Events

Instances

Instances

Instance Types

Launch Templates

Spot Requests

Savings Plans

Reserved Instances

Dedicated Hosts

Capacity Reservations

Capacity Manager

Images

AMIs

AMI Catalog

Elastic Block Store

Volumes

Snapshots

Lifecycle Manager

Network & Security

Security Groups

Elastic IPs

Placement Groups

Key Pairs

Network Interfaces

Load Balancing

sg-04bc7780b94197ddf - web-sg

Actions

Details

Security group name

web-sg

Security group ID

sg-04bc7780b94197ddf

Description

allow access to different group of people

VPC ID

vpc-0d396deeed198cbcf

Owner

902132373241

Inbound rules count

3 Permission entries

Outbound rules count

1 Permission entry

Inbound rules

Outbound rules

Sharing

VPC associations

Related resources - new

Tags

Inbound rules (3)

Manage tags

Edit inbound rules

Search

☐	Name	Security group rule ID	IP version	Type	Protocol	Port range	Source
☐	-	sg-067fde5eebf2f1136	IPv4	SSH	TCP	22	182.48.231.129/32
☐	-	sg-08c2b19669f962192	IPv4	HTTP	TCP	80	0.0.0.0/0
☐	-	sg-0c3277571abb807c8	IPv4	Custom TCP	TCP	5000	0.0.0.0/0

VPC dashboard < sg-0a5686b9ac6aad5cf - db-sg1 Actions

AWS Global View [↗](#)

Filter by VPC: ▾

Virtual private cloud

Your VPCs

Subnets

Route tables

Internet gateways

Egress-only Internet gateways

Carrier gateways

DHCP option sets

Elastic IPs

Managed prefix lists

NAT gateways

Peering connections

Route servers

Security

Network ACLs

Security groups

PrivateLink and Lattice

Getting started

Endpoints

Endpoint services

Service networks

Details

Security group name db-sg1

Security group ID sg-0a5686b9ac6aad5cf

Description allow db

VPC ID vpc-098cca9190def8507

Owner 902132373241

Inbound rules count 1 Permission entry

Outbound rules count 1 Permission entry

Inbound rules Outbound rules Sharing VPC associations Related resources - new Tags

Inbound rules (1)

Search

Manage tags Edit inbound rules

☐	Name	Security group rule ID	IP version	Type	Protocol	Port range	Source
☐	-	sg-032224326fba3661	IPv4	MySQL/Aurora	TCP	3306	0.0.0.0/0

☐ IAM role with attached policies

IAM > Roles > blocksupply-ec2-role

Identity and Access Management (IAM)

Search IAM

Dashboard

Access Management

Roles

Policies

IAM users

IAM user groups

Identity providers

Account settings

Root access management

Temporary delegation requests

Access reports

Access Analyzer

Resource analysis

Unused access

Analyzer settings

Credential report

Organization activity

Service control policies

Resource control policies

Related consoles

IAM Identity Center [↗](#)

AWS Organizations [↗](#)

blocksupply-ec2-role info

Allows EC2 instances to call AWS services on your behalf.

Summary

Creation date June 19, 2026, 20:54 (UTC+05:30)

ARN arn:aws:iam::902132373241:role/blocksupply-ec2-role

Instance profile ARN arn:aws:iam::902132373241:instance-profile/blocksupply-ec2-role

Last activity 15 minutes ago

Maximum session duration 1 hour

Permissions Trust relationships Tags Last Accessed Revoke sessions

Permissions policies (2) info

You can attach up to 10 managed policies.

Filter by Type All types

☐	Policy name	Type	Attached entities
☐	AmazonS3FullAccess	AWS managed	2
☐	CloudWatchAgentServerPolicy	AWS managed	1

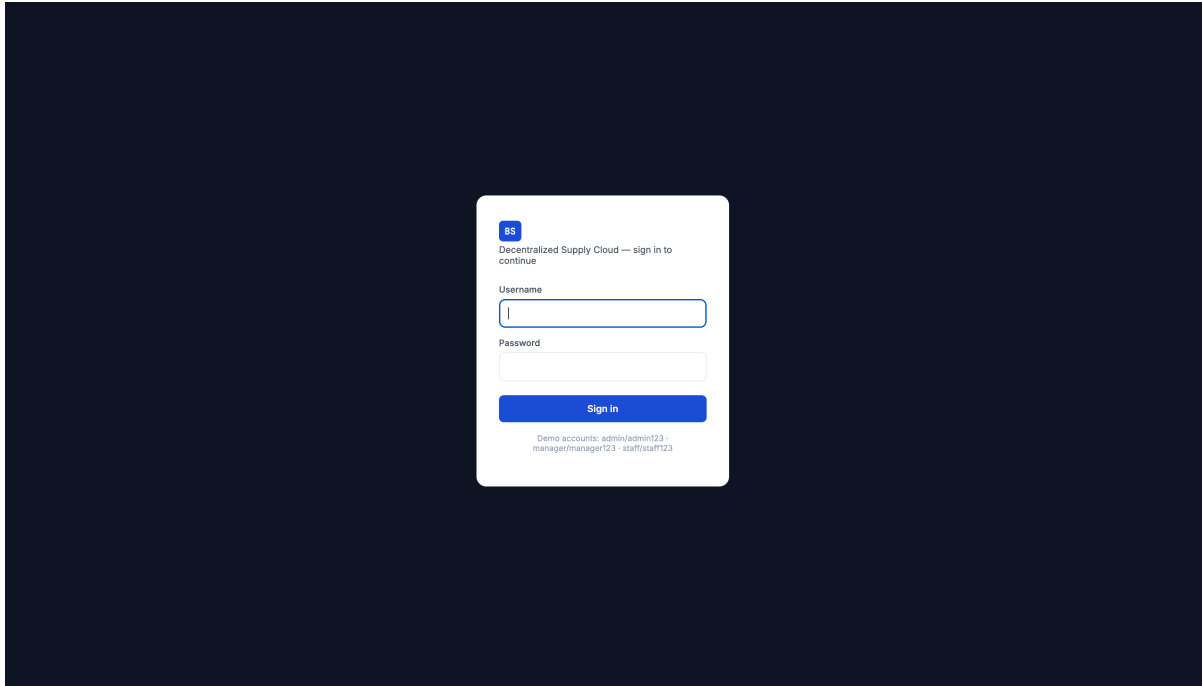
Permissions boundary (not set)

Generate policy based on CloudTrail events

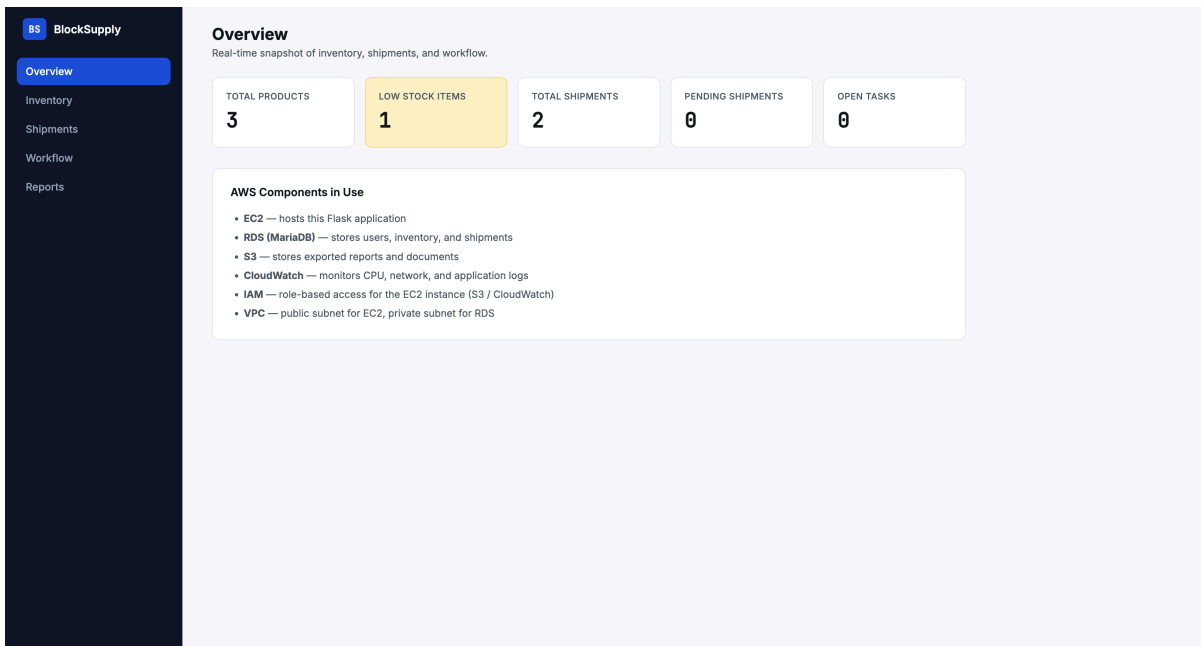
You can generate a new policy based on the access activity for this role, then customize, create, and attach it to this role. AWS uses your CloudTrail events to identify the services and actions used and generate a policy. [Learn more](#)

Generate policy

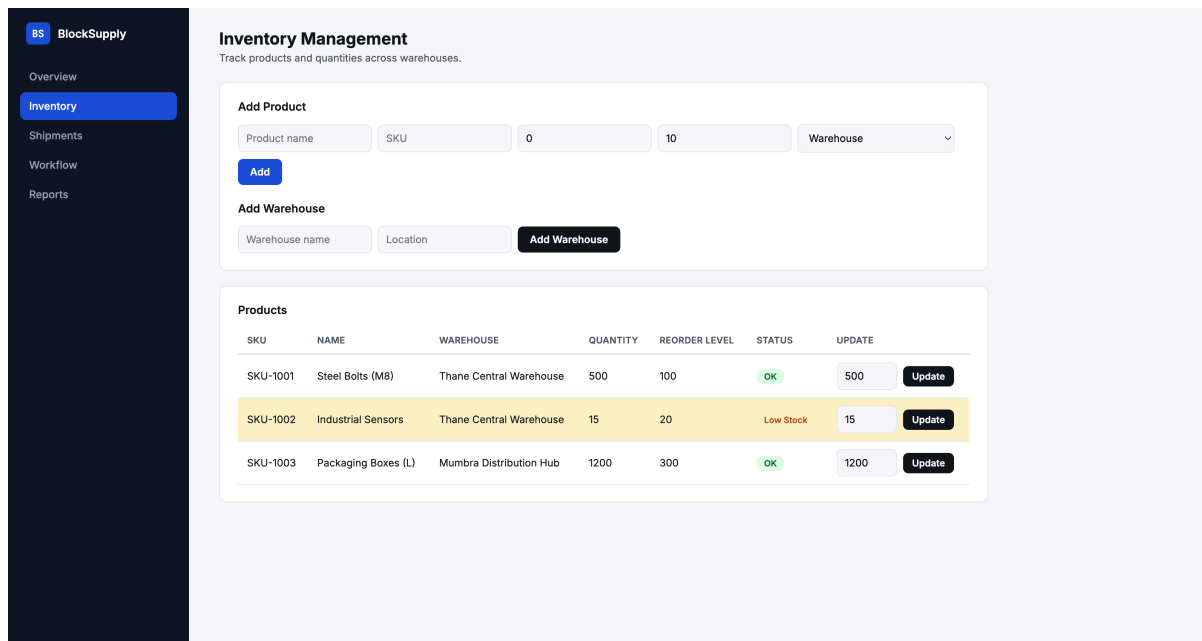
☐ App — Login page



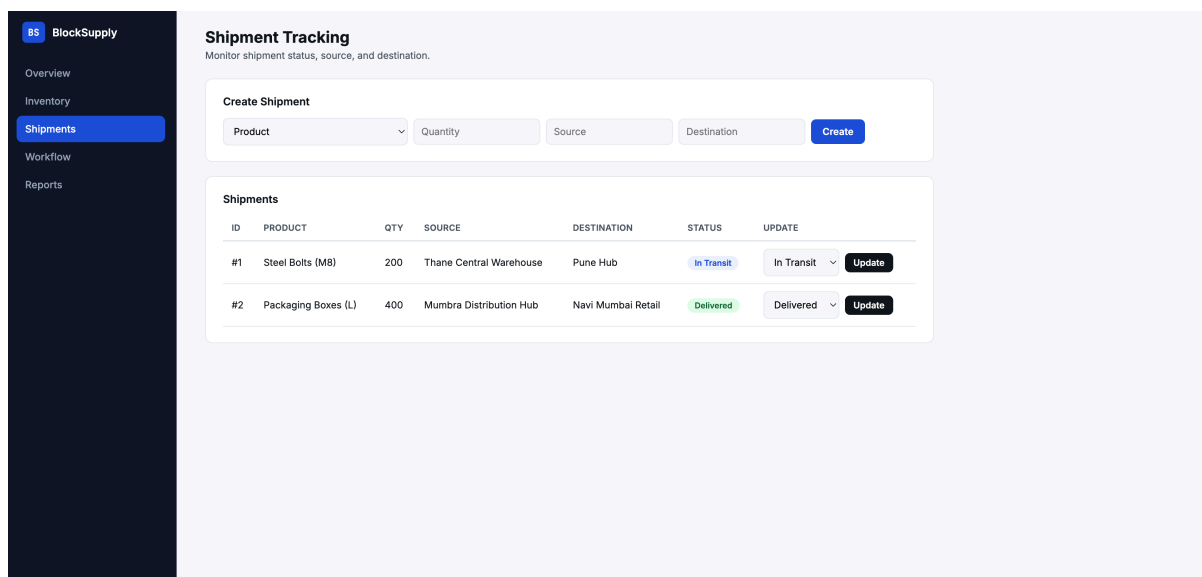
☐ App — Dashboard / Overview



☐ App — Inventory management



□ App — Shipment tracking



□ App — Workflow management

BS BlockSupply

Overview

Inventory

Shipments

Workflow

Reports

Workflow Management

Assign tasks and route them through approval chains.

Assign Task

Task title

Description

Assignee

dd/mm/yyyy

Assign

Tasks created by a manager automatically route to an admin for approval.

Tasks

TITLE	ASSIGNEE	DUE	STATUS	UPDATE
-------	----------	-----	--------	--------

Pending Approvals

TASK	REQUESTED BY	DECISION
------	--------------	----------

App — Reports page

BS BlockSupply

Overview

Inventory

Shipments

Workflow

Reports

Reporting Dashboard

Inventory metrics and shipment analytics.

Inventory by Warehouse

WAREHOUSE	TOTAL QUANTITY
Thane Central Warehouse	515
Mumbra Distribution Hub	1200

Shipment Status Breakdown

STATUS	COUNT
In Transit	1
Delivered	1

Low Stock Alerts

SKU	NAME	QUANTITY	REORDER LEVEL
SKU-1002	Industrial Sensors	15	20

Export

Generate a CSV inventory report, upload it to S3, and download a copy.

Export to S3 + Download